Concorrenza

Message Passing

Paradigmi

P.sincronizzazione
usa memorie condivise:
semafori, monitor

P.comunicazione
usa messaggi:
message passing

Definizioni

messaggi: informazioni
formattare da mitt per dest

scambio di messaggi:
copia messaggio tra spazi
d'indirizzamento M/D
(gestito dall'os)

operazioni:
* send (dest noto)
* receive (mitt (non)noto)

Tassonomia

sincrono:
send sincrono e
receive bloccante

asincrono:
send asincrono
e receive bloccante

completamente asincrono:
send asincrono
e receive non
bloccante

Problemi classici di acesso esclusivo a risorse

producer/consumer

Attori:
* producer (P), genera e traferisce (W)
* consumer (C), prende e consuma (R)

Strutture:
* Una variabile condivisa di "valori"
* semafori binari "full" e "empy"

Proprietà:
* P/C attende di W/R finchè
l'altro non ha finito
* +no deadlock

bounded buffer

Attori=producer/consumer

Struture:
* Vettore circolare condivso
di "valori",
front idx W e rear idx R
* semafori "full" e "empy"
range=len(vettore)

Proprietà:
* =produce/consumer
* +no starvation

dinig philosophers (asian)

Attori: 5chopsticks, 5filosofi, alternano:
* pensare (0 chopstick)
* mangiare (2chopstick di 5)

competizione risorse comuni
(P/C, BB cooperavano)

Strutture: vettore chopsticks sono
semafori binari

filosofo prende chopstick alla sua
sinistra e poi destra, (1 filosofono
mancino risolve deadlock)

Proprietà: riscio deadlock e starvation

Sincronizzazione

supporto OS/linguaggio

Semafori e Monitor:
comunicazione tramite
memoria proc. condivisa

Message Passing:
comunicazione diretta
tra proc.

Semafori

Definizione

IDEA: proc. bloccato in specifici
punti, CS, in atteda di segnali
da altri proc.

REALIZZAZIONE: segnale->
tipo astratto semaforo:
* campo int i (inizializzato≥)
* operazioni atomiche:
** V, invia segnale,
(rilascio risorsa)
i++
** P, attende segnale
(attendi e prendi risorsa)
if(i≥0) then i--

P e V delimitano la CS

Processi in attesa

S.D. coda FIFO proccessi
attesa P (sospesi),
fair (no starvation)

si starvation se non
definisco politica
"risveglio"

busy-waiting RARO
in P e V

Sistem-Call

suspend(pidProcess)
mette il proc. in waiting

wakeup(pidProcess)
mette il proc. in ready

Binari

i=0/1, programmi semplici e si muta esclusione

Cos'è

Modello

+processi eseguiti in
parallelo reale/apparente

concorrenza:
tecniche risolvono
problemi di
comunicazione e
sincronizzazione

usata per:
* applicazioni multiple
* App. strutturate
su processi
* Strutture del OS

Processi

attività controllata
da programma
svolta su cpu
(è dinamico)

assioma finite progress:
velocità finita
ma sconosciuta

programma:
* statico
* sequenza istruzioni
* no tempo esecuzione

1programma +processi

Stato processo

img di memoria:
dati + SD del OS

img nel processore
registri generali e speciali

stato d'avanzamento

Processi multipli

Multiprogramming:
1cpu +processi
+-parallelismo
(alternati nel tempo,
interleaving)

Multiprocessing:
1pc +cpu +processi
+parallelismo
(alternati nello spazio,
overlapping)
(multiprogramming
per cpu)

Distributed processing:
+pc +cpu +processi
+parallelismo

Problemi comuni

* impossibile predire
istanti esecuzione proc.
* accesso risorse
condivise

-Race condition:
risultato dipende
temporizzazione
processi

Interazioni tra processi

Tipi operazioni

consapevolezza altri proc.:
* Ignari: non fatti per
cooperare (abiente comune),
competono per risorse,
os arbrità
* Indiretti: risorse condivise
per scambi dati,
scopo comune,
SC del OS sincronizzano
* Diretti: conoscono id,
scambiano messaggi,
scopo comune,
SC del OS aiutano

Proprietà

def.: attributo vero
per ogni possibile
esecuzione del prog.

* safety,
accesso risose "civile"
-race condition
* liveness,
sincronizzazione non
deve bloccare proc.

Termini

Mutua esclusione:
risorsa accessibile ad 'un
solo proc. per istante

Deadlock:
blocco processo per
risorse circolarmente
occupate da proc.
(definitivo)

Starvation:
inaccessibilità risorsa
perchè "sempre occupata"
(temporaneo)

Azioni atomiche

compiute in modo
indivisibile, parallelismo:
* reale, no interferenze
altri proc.
(int prima o dopo)
+arbritaggio bus risolve
concorrenza risorse
* apparente, context switch
prima o dopo azione
(int mai durante)

indicate con <S>

Sezioni Critiche

Cosa sono

zone programma speciali
da eseguire in modo atomico

usano risorse condivise

evitare deadlock o starvation

[enter cs]...[exit cs]

una cs va liberata
(proc. terminano fuori sc)

Requisiti

* mutua esclusione
* no deadlock
* no delay superflui
* no starvation

Modalità

software
proc. accesso a
oggetto distribuito,
+errori,
busy waiting: stato
proc. in polling variabili

hardware (interrupt)
usa istruzioni speciali
efficente, no
general-purpose